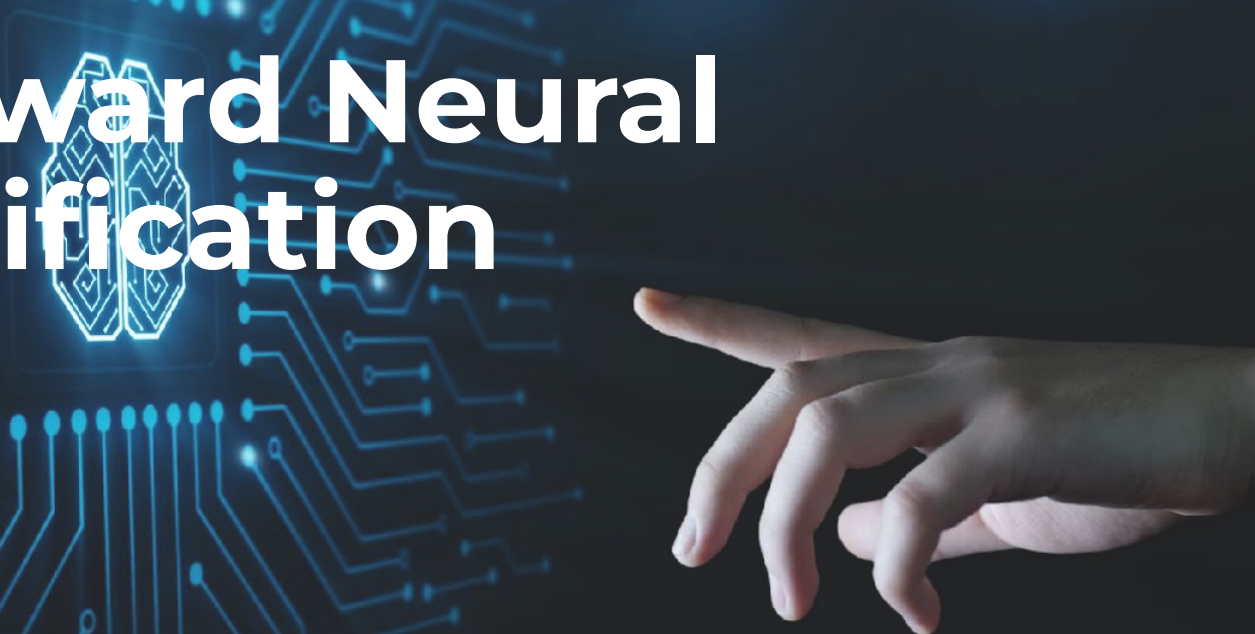




EMERITUS

Learn. From the world's best.

Multilayer Feedforward Neural Networks for Classification and Regression

A hand is shown on the right side of the image, pointing its index finger towards a glowing, stylized brain icon. The brain is composed of geometric patterns and is surrounded by a complex network of glowing blue lines that resemble a circuit board or neural pathways. The background is dark with a subtle blue glow.

Recap: Logistic Regression to MLPs

This session builds on your neural network basics. You've already covered:

Perceptrons vs. activation functions

Feedforward architecture

Output activations: sigmoid (binary), softmax (multiclass)

Gradient descent & backpropagation

Recap: Logistic Regression to MLPs

Today's Implementation Objectives



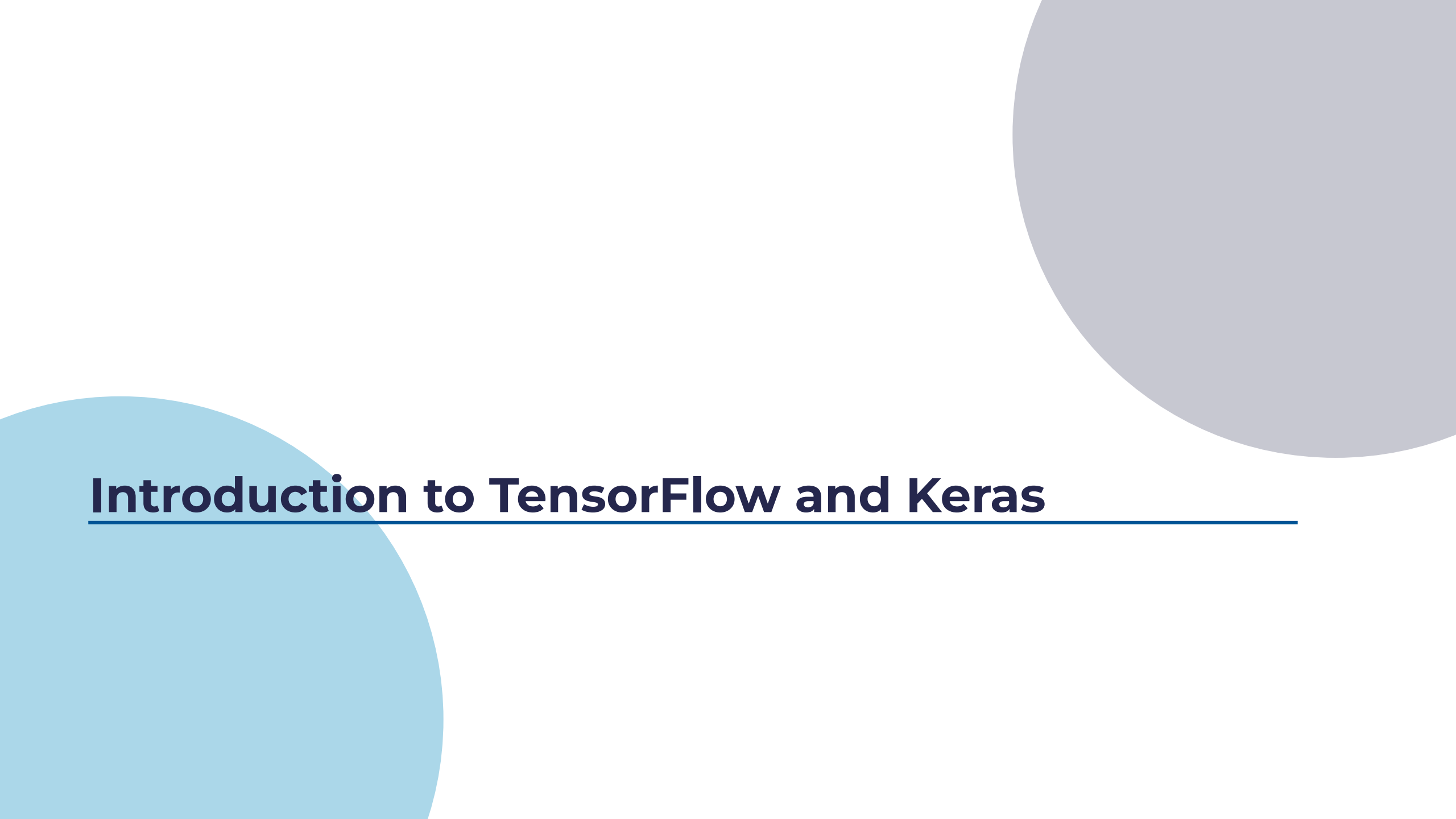
Binary Logistic Regression: Build and test a simple binary classifier in Keras



Custom Training Loop: Use `tf.GradientTape` to track gradients and updates



Multiclass Classification: Train a softmax model on Fashion-MNIST with one linear layer



Introduction to TensorFlow and Keras

What is TensorFlow?

Open-source ML Platform By Google (2015)

TensorFlow is an open-source ML platform by Google (2015) that offers tools, libraries, and resources to build and deploy machine learning applications.

TensorFlow provides easy APIs for beginners and detailed control for experts, making it adaptable for all skill levels and use cases



- Deploy across desktop, mobile, web and cloud
- Visualise with TensorBoard tools
- Accelerate using GPUs and TPUs
- Scale production-ready models

What is Keras?

High-level Neural Networks API Integrated With Tensorflow In 2017

Keras is a high-level neural networks API, integrated into TensorFlow in 2017, designed for fast, user-friendly, and modular deep learning experimentation



User-Friendly Design

- Provide a simple, consistent API for people
- Reduce cognitive load with clear error messages

Multi-Backend Support

- Support TensorFlow, JAX, and PyTorch backends
- Enable code portability across frameworks

Industry Adoption

- Use widely from startups to Fortune 500 companies
- Enable rapid prototyping and production deployment

Why Use TensorFlow with Keras?

Combine Keras' ease of use with TensorFlow's power and scalability



Rapid Prototyping: Build and iterate models quickly with Keras' simple API



Performance Optimisation: Use TensorFlow's C++ backend with GPU/TPU acceleration



Production Deployment: Move seamlessly from research to production with TF Serving, TF Lite, and TensorFlow.js



Flexible APIs: Choose from Sequential, Functional, or Subclassing for simple or complex models

Get the best of both: Keras' ease with TensorFlow's power and ecosystem

Core Concepts: Neural Networks in Keras

Key Building Blocks And Training Components

Building Blocks

- Layers – Dense, Conv2D, LSTM, Dropout, BatchNormalization, and more
- Activation Functions – ReLU, sigmoid, tanh, softmax for non-linearity

Training Components

- Loss Functions – Categorical crossentropy (classification), MSE (regression), custom losses
- Optimisers – Adam, SGD, RMSprop, and others for weight updates

These components shape neural networks to learn patterns and make predictions

Real-World Applications of TensorFlow & Keras

Powering Innovation Across Industries With Ai-driven Solutions



Healthcare

Early disease detection,
patient monitoring,
drug discovery,
personalised treatment



Natural Language Processing

Translation, sentiment
analysis, chatbots,
summarisation, voice
assistants



Autonomous Systems

Self-driving cars,
robotics, drones,
predictive
maintenance, quality
control

These applications show the versatility of TensorFlow and Keras, used from startups to tech giants to tackle complex problems and drive innovation

Getting Started Today

Install Easily, Meet Requirements and Explore Learning Resources

Quick Installation

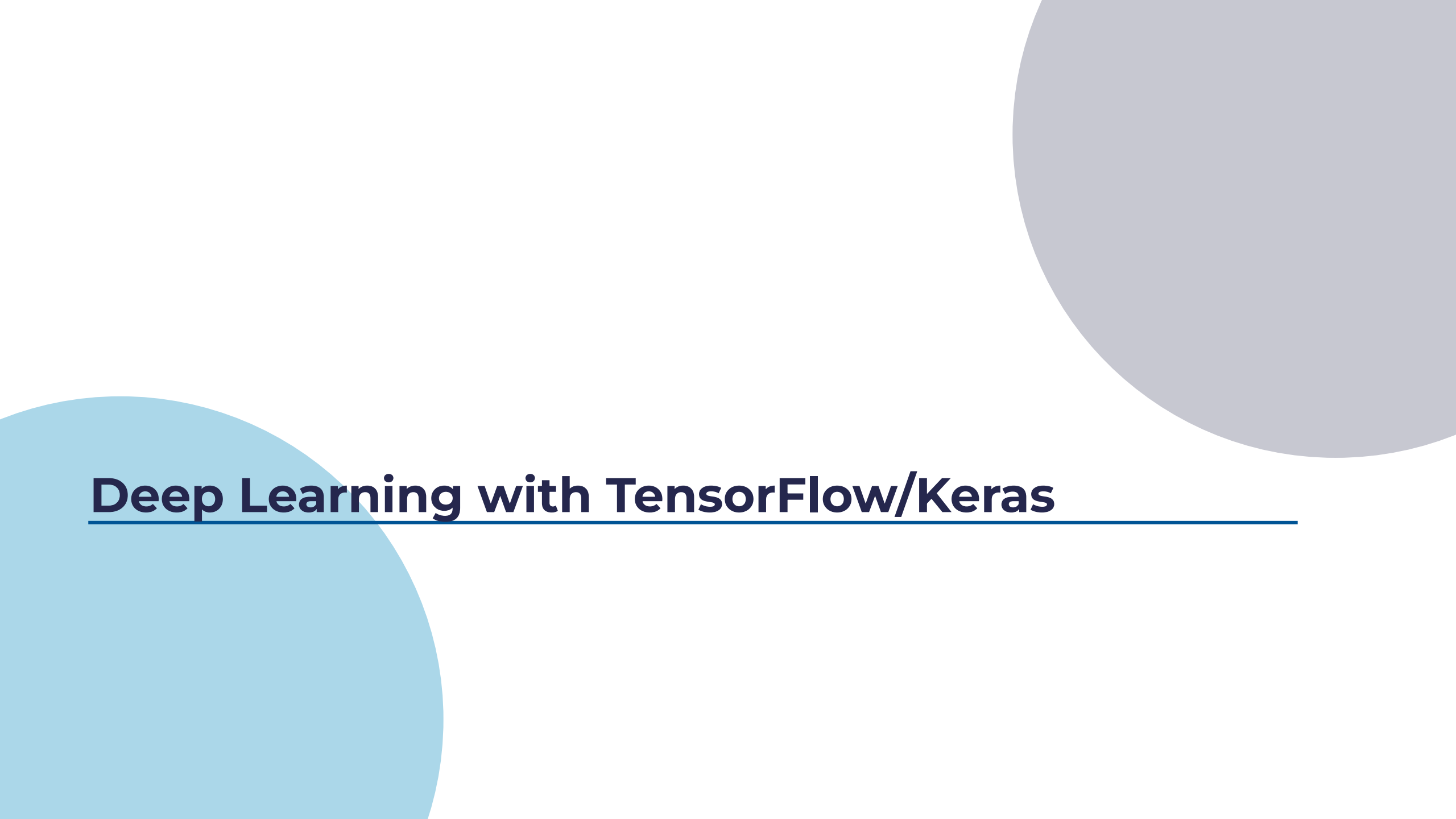
```
# CPU-only version  
pip install tensorflow  
# With GPU support (CUDA required)  
pip install tensorflow[gpu]  
# For Jupyter notebooks  
pip install jupyter matplotlib
```

System requirements: Python 3.7–3.10,
GPU drivers (for GPU use), 2GB+ RAM

Learning Resources

- Official docs: [tensorflow.org](https://www.tensorflow.org), keras.io
- Interactive TensorFlow tutorials
- Keras examples with code samples
- TensorFlow Developer Certificate
- Community forums & GitHub discussions

Start simple, progress to complex projects with
community support



Deep Learning with TensorFlow/Keras

Building on Your Async Learning

Applying Core Neural Network Concepts

In the async materials, you explored:

- Perceptrons vs continuous activations
- Sigmoid, tanh, ReLU
- Softmax for multiclass
- Applications in vision, text, audio

Today we'll implement these concepts:

- Feedforward network forward pass
- Loss functions & optimisation
- Gradient descent & backpropagation
- Training modes & techniques

Environment and Datasets

Set Up Tools And Use Sample Datasets For Practice

Development Environment

```
pip install -U tensorflow scikit-learn matplotlib  
import numpy as np  
import matplotlib.pyplot as plt  
import tensorflow as tf  
from sklearn.datasets import make_classification  
from sklearn.model_selection import train_test_split  
from sklearn.preprocessing import StandardScaler
```

Environment and Datasets

Set Up Tools And Use Sample Datasets For Practice

Datasets for Live Coding

- **Binary Classification:** *make_classification* with 2 informative features, ideal for decision boundaries
- **Multiclass Classification:** *tf.keras.datasets.fashion_mnist* or *mnist*
Fashion MNIST or MNIST (10 image classes, 28×28 pixels)

Requirements: Python 3.10+, TensorFlow 2.x (with Keras API),
NumPy, scikit-learn, matplotlib

Warm-up: Binary Logistic Regression with Keras

Let's start with a high-level implementation:

```
# Create dataset
X, y = make_classification(n_samples=4000, n_features=2, n_informative=2, n_redundant=0, n_clusters_per_class=1, random_state=42)

# Train-test split
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2)

# Standardize features
sc = StandardScaler()
X_tr = sc.fit_transform(X_tr)
X_te = sc.transform(X_te)

# Build model
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(2,)),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

# Compile model
model.compile(optimizer=tf.keras.optimizers.SGD(learning_rate=0.1), loss='binary_crossentropy', metrics=['accuracy'])
```


Warm-up: Binary Logistic Regression with Keras

```
# Train model
history = model.fit(x_tr, y_tr, epochs=30, batch_size=64, validation_split=0.2, verbose=0)

# Evaluate model
loss, acc = model.evaluate(x_te, y_te, verbose=0)
print(f"Test Acc: {acc:.3f}")
```

Under the Hood: Gradient Descent

Parameter Update Rule

$$W \leftarrow W - \eta \nabla_w \mathcal{L}$$

Where:

- w : model weights
- η : learning rate
- $\nabla_w \mathcal{L}$: gradient of loss with respect to weights

Training Modes

1. **Batch:** All data, stable but slow
2. **Stochastic (SGD):** Per sample, fast but noisy
3. **Mini-batch:** Mix of both, common choice

Custom Training Loop with tf.GradientTape

See the gradients explicitly with TensorFlow's automatic differentiation:

```
import numpy as np
import tensorflow as tf
# Define model
model = tf.keras.Sequential([
    tf.keras.layers.Input((2,)),
    tf.keras.layers.Dense(1, activation='sigmoid')])

# Optimizer & loss
opt = tf.keras.optimizers.SGD(1e-1)
bce = tf.keras.losses.BinaryCrossentropy()
```

Custom Training Loop with tf.GradientTape

```
# Custom training step
@tf.function
def train_step(xb, yb):
    with tf.GradientTape() as tape:
        preds = model(xb, training=True)
        loss = bce(yb, preds)
    grads = tape.gradient(loss, model.trainable_variables)
    opt.apply_gradients(zip(grads, model.trainable_variables))
    return loss

# Training loop
for epoch in range(20):
    idx = np.random.permutation(len(X_tr))
    for i in range(0, len(X_tr), 64):
        batch = idx[i:i+64]
        _ = train_step(X_tr[batch], y_tr[batch])
```

This custom loop gives full control of training and gradient inspection

Multiclass Logistic Regression with Softmax

Moving from binary to multiclass classification with Fashion-MNIST:

```
import tensorflow as tf
# Load and preprocess data
(x_tr, y_tr), (x_te, y_te) = tf.keras.datasets.fashion_mnist.load_data()
x_tr = x_tr.reshape(-1, 28 * 28) / 255.0
x_te = x_te.reshape(-1, 28 * 28) / 255.0

# Build model with softmax for 10 classes
model = tf.keras.Sequential([
    tf.keras.layers.Input((784,)),
    tf.keras.layers.Dense(10, activation='softmax')
])
# Compile with categorical cross-entropy
model.compile(
    optimizer='sgd',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)
# Train
model.fit(x_tr, y_tr, epochs=10, batch_size=128, validation_split=0.1, verbose=0)
# Evaluate
```

Backpropagation Intuition



Forward Pass

$$z^{(l)} = w^{(l)} h^{(l-1)} + b^{(l)}$$

$$h^{(l)} = \phi(z^{(l)})$$

Where ϕ is the activation function
(ReLU, sigmoid, etc.)

Backward Pass

1. Compute gradient of loss
2. Propagate gradients backwards
3. Accumulate gradients for weights and biases
4. Update parameters with optimiser

TensorFlow automates this process, but understanding it aids debugging and optimisation

Diagnostics & Debugging Neural Networks



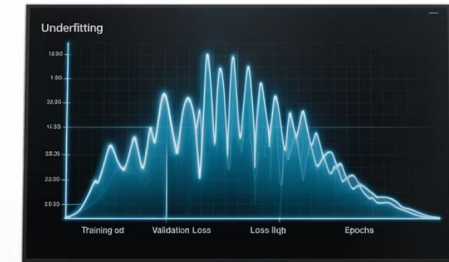
Loss Not Decreasing

- Adjust learning rate
- Check data scaling/normalisation
- Verify label format (int vs one-hot)
- Match activation with loss function



Overfitting

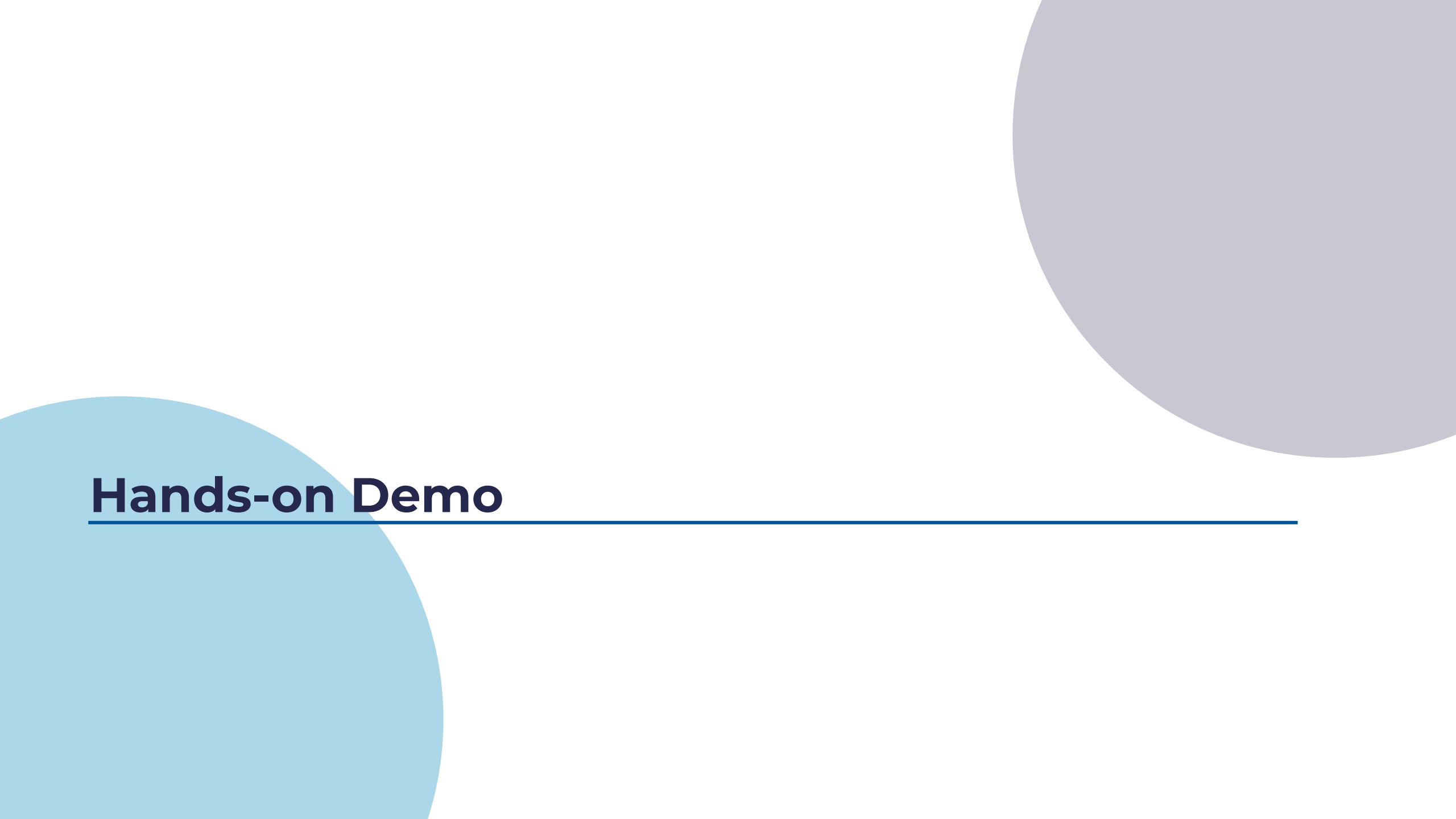
- Add regularisation (dropout, L2)
- Use early stopping
- Apply data augmentation
- Reduce model size



Underfitting

- Increase layers/neurons
- Train longer (more epochs)
- Try learning rate warmup
- Improve feature engineering

Class Imbalance: Use class weights or resampling (over/under-sampling)



Hands-on Demo

Q&A Session



Open floor for questions and clarifications.